**PARCOURS L3 EEA**
**PROJET EEA**
**Année 2024-2025**

# ROBOTIQUE MOBILE :
## Stabilisation de Mouvement

**Projet présentée par :**

**AKAMGA KAMGA Loïc**

Table des matières

**1- GENERALITE SUR LA ROBOTIQUE** ..... 2
**1.1-Les robots à bases fixe** ..... 2
**1.2- Les robots à base mobile** ..... 3
**1.3- Cas particulier des robots mobiles à roue** ..... 4
**1.4-Problématique** ..... 5
**2-OBJECTIF DU PROJET** ..... 6
**3- INTERFACE DE SIMULATION** ..... 6
**3.1-Fonctionnement de l'interface** ..... 7
**3.2- Communication Coppeliasim-Python** ..... 8
**3. 3- Configuration de la scène** ..... 9
**4- DESCRIPTION DU PIONEER 3DX** ..... 10
**5 - Première approche : commande proportionnelle** ..... 10
**5.1 - Introduction à la commande proportionnelle et au modèle cinématique** ..... 10
**5.2 - Principe du suivi suiveur-référence** ..... 12
**5.3 - Loi de commande proportionnelle** ..... 13
**5.4 - Description de la méthode du triangle** ..... 14
**5.5 - Résultats des simulations** ..... 14
**6. Commande par retour d'état sur l'erreur de posture** ..... 18
**6.1. Principe de la commande** ..... 18
**6.2. Application de la commande sous CoppeliaSim** ..... 19
**6.3. Les Données** ..... 20
**6.4. Simulation et Resultats** ..... 20
**6.5. Comparaison** ..... 24
**7. Conclusion** ..... 28
**Annexe** ..... 29
**Annexe A – Code Python Loi de Commande proportionnel** ..... 29
**Annexe B – Code Python Loi de Commande 2** ..... 31

# 1- GENERALITE SUR LA ROBOTIQUE

Un robot est une machine programmable qui se caractérise par sa capacité à percevoir, à prendre des décisions et à interagir avec son environnement. En fonction de la mobilité du robot on distingue deux grands types de robot :

## 1.1-Les robots à bases fixe

Ce sont des robots qui possèdent un environnement de travail bien défini et sont incapables de se déplacer dans leur espace de travail. On les retrouve le plus souvent dans les domaines industriel et médical. On leur attribue aussi l'appellation de manipulateur. La figure 1.1 montre un robot manipulateur utilisé en médecine et la figure 1.2 montre un bras robot industriel.

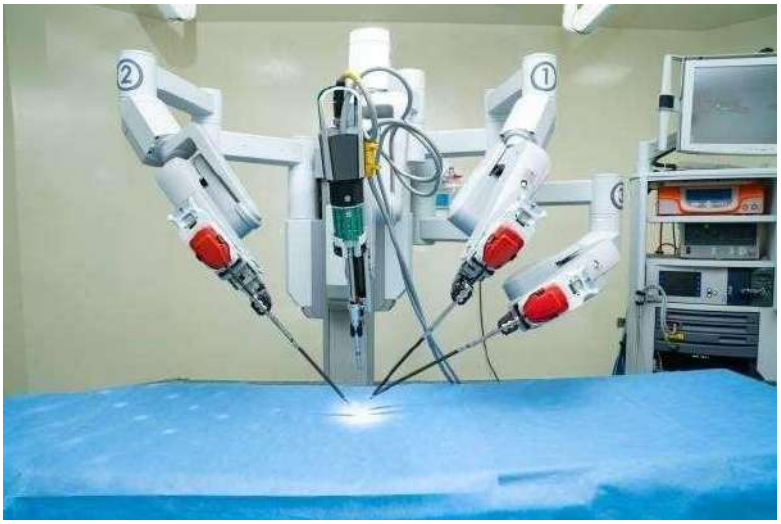

***Figure 1.1 : robot de télé-chirurgie source : (istock média)***

***Figure 1.2: Bras de robot industriel (source : Freepik)***

## 1.2- Les robots à base mobile

Contrairement aux robots à bases fixes, les robots mobiles, en plus de la perception, ont la capacité de naviguer, voire de se localiser dans leur environnement de travail. Dans cette catégorie, on peut distinguer les robots terrestres (à roues, comme illustré à la figure 1.2.1A, et à pattes, figure 1.3), ainsi que les robots aériens (figure 1.2.1B) et sous-marins (figure 1.2.1C). Ils sont utilisés dans le domaine de l'exploration spatiale, la surveillance, l'agriculture et la médecine.

A B C

Figure 1.2.1: Robot à base mobiles[source:(A) -Nasa 2020 ; (B)-Futura science ; (C)-Ecoptik]

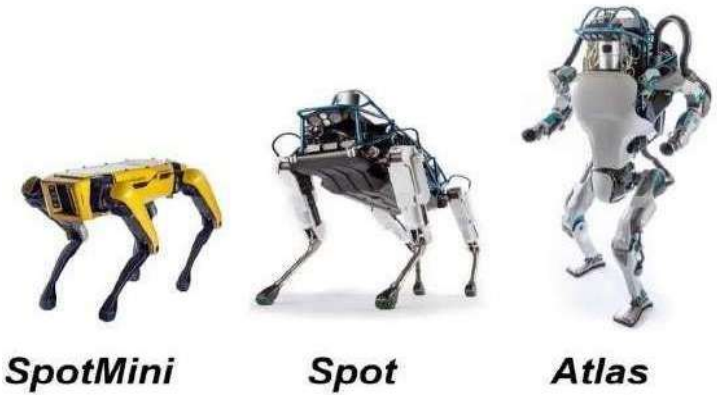


Figure 1.3 : Robots mobiles à pattes (source : Boston Dynamic)

## 1.3- Cas particulier des robots mobiles à roue

Les robots mobiles à roues présentent une certaine contrainte liée à la manière dont les roues interagissent avec le sol. On parle de contrainte holonome ou non holonome.

- Robots mobiles à roues holonomes : ils se caractérisent ou se distinguent par leur capacité à effectuer des mouvements latéraux, ce qui leur permet de se déplacer dans toutes les directions possibles du plan. La figure 1.3.1A montre un robot à roue holonome et la figure 1.3.1B une roue holonome.

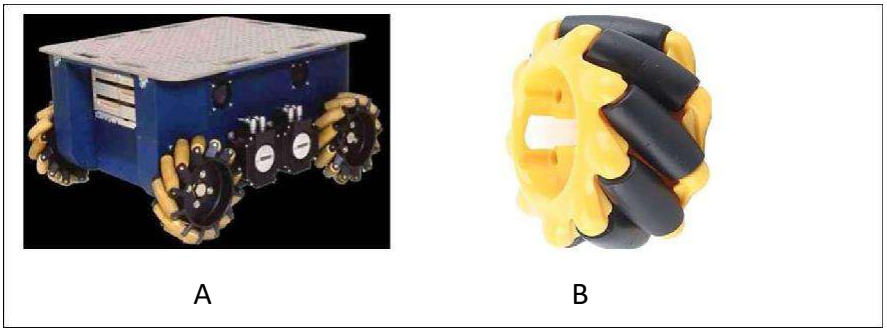


Figure 1.3.1 :(A) - robot à roue holonome (B)- roues holonome mecanum (source : Wikipédia)

- Robots mobiles à roues non holonomes : leur déplacement dans le plan est limité par rapport aux mobiles holonomes. La figure 1.3.2A et 1.3.2B montre respectivement un robot non holonome et une roue non holonome.

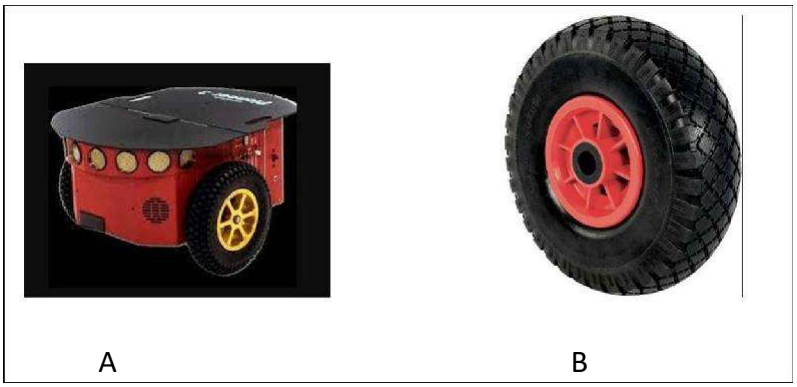


Figure1.3.2:(A) - robot non holonome) ;(B) roue holonome (source :[2]

**REMARQUE** : Différence fondamentale entre robot mobile holonome et non holonome

Cette différence se matérialise par le type de déplacement effectué par chaque type de robot. C'est le cas du robot holonome qui peut se déplacer latéralement sans changer d'orientation, grâce aux rouleaux inclinés présents sur les roues. La figure 1.3.4A montre les déplacements d'un mobile holonome, tandis que la figure 1.3.4B montre les déplacements d'un mobile non holonome.

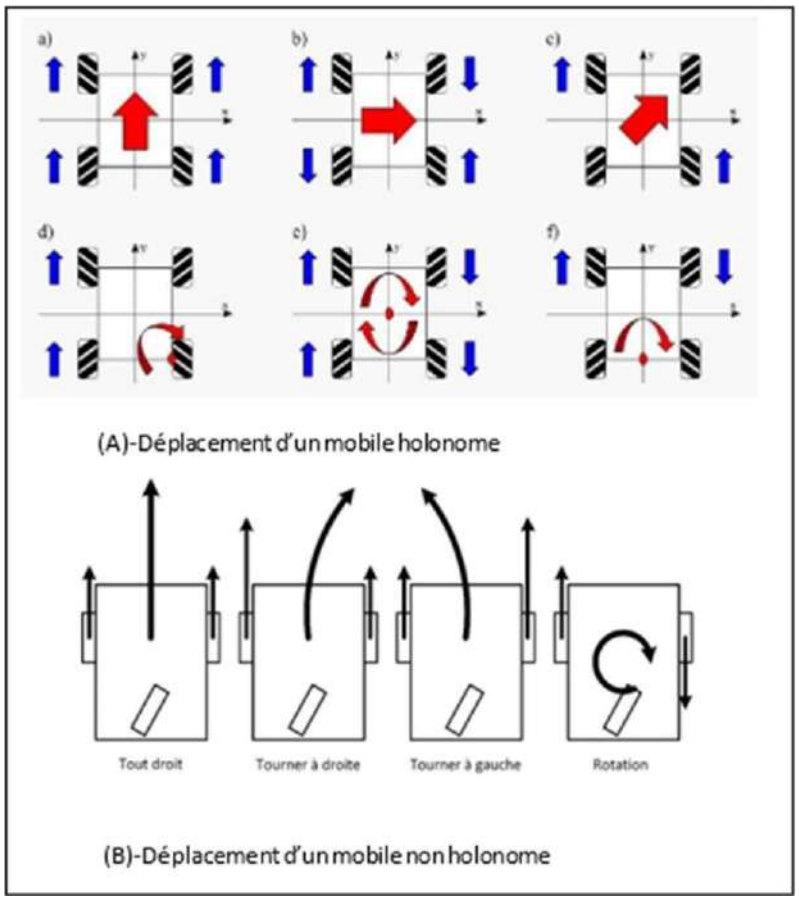


Figure1.3.4:(A) - source (Castel'Lab le Fablab MJC de Château-Renault) ;(A) - source : [2 ]

## 1.4-Problématique

Avant pour poursuivre, il est fondamental de formuler de manière rigoureuse le problème de la stabilisation du mouvement des robots, afin d'en préciser les enjeux et les contraintes associées.

Considérons deux robots mobiles à roues non holonomes dont le premier (référent) suit une trajectoire planifiée et le second (suiveur) doit se contenter de reproduire exactement la

trajectoire du référent. On peut alors définir l'exécution du mouvement du robot suiveur comme sa capacité à reproduire en temps réel la trajectoire du robot référent. Cependant, de nombreuses contraintes (non-holonomie, environnement, changement de vitesse, erreur de mesure des capteurs, lois de commande) empêchent le robot suiveur de reproduire exactement la trajectoire du référent.

Ainsi on se demande comment contrôler ou corriger en temps réel la trajectoire du suiveur pour qu'il reproduise exactement celle du référent.

## 2- OBJECTIF DU PROJET

LE principal objectif de ce projet est de réussir à stabiliser la trajectoire du suiveur par rapport à celle du référent. C'est-à-dire de trouver un modèle mathématique ou une loi qui permette de corriger (réduire) l'écart ainsi que les erreurs d'orientation entre la trajectoire du suiveur et celle du référent.

Pour tester expérimentalement la loi de commande la loi de commande établi CoppeliaSim est choisi comme moteur de simulation et de visualisation. Ce logiciel est largement utilisé dans le domaine de la robotique.

Les contributions sont repartir comme suite :

| Nom et prénom | Contributions |
|---|---|
| Maram El Atrach | Première approche : commande proportionnelle |
| Jonathan Mabon Fonseca | Commande par retour d'état sur l'erreur de posture |
| Loic Akamga Kamga | Connexion entre CoppeliaSim et Python et paramètres de l'interface |

## 3- INTERFACE DE SIMULATION

CoppeliaSim présente de nombreux avantages pour la simulation des robots,

notamment une large gamme de robots intégrés dans sa bibliothèque ; des outils de mesure, et surtout la communication avec d'autres logiciels ou ordinateurs.

## 3.1-Fonctionnement de l'interface

L'interface est principalement constituée d'une scène qui permet de visualiser les objets ainsi que la simulation en 3D. Le navigateur de modèles permet d'utiliser des robots préfabriqués. La console montre l'état et l'évolution de la simulation en temps réel. La figure 3.1.1 montre un aperçu du moteur de simulation CoppeliaSim.

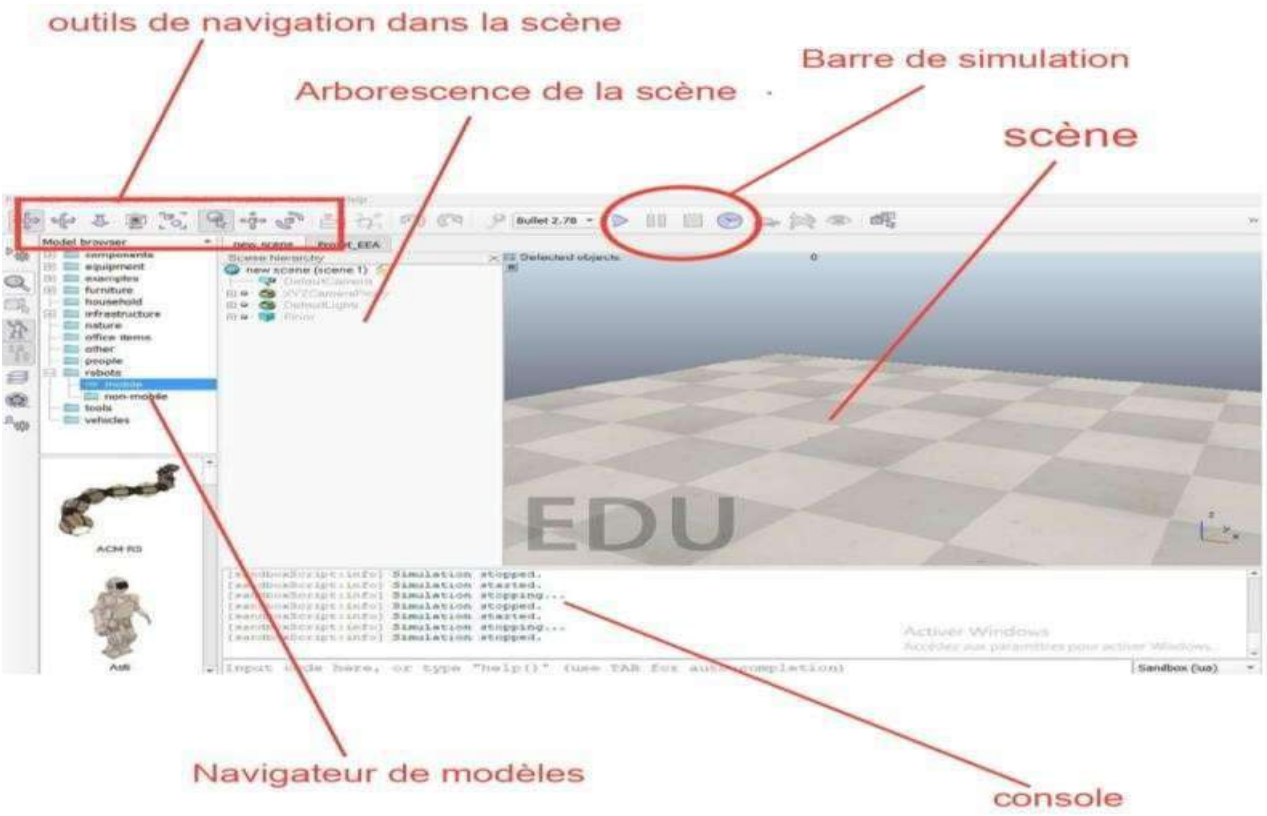


Figure 3.1.1 : moteur de simulation Coppeliasim

## 3.2- Communication Coppeliasim-Python

L'une des particularités de ce simulateur est que la simulation peut être lancée et contrôlée depuis un autre logiciel ou ordinateur. Cela est possible grâce à l'interface de programmation d'application (API).

L'API est le terme utilisé pour toute communication entre deux processus, c'est une méthode qui permet à deux logiciels (programmes) de communiquer entre eux. Les API possèdent un ensemble de fonctions, comme le ZMQ (Zero Message Queue), qui est utilisé spécifiquement dans le cas de la liaison entre Python et CoppeliaSim qu'on surnomme « zmq Remote API ».

Autrement dit la bibliothèque ZMQ se comporte comme le canal de communication entre le script python et le logiciel CoppeliaSim. Les figures (2.2.1) ; **(3.2.2) ;** (3.2.3) montre la liaison entre python-CoppeliaSim.

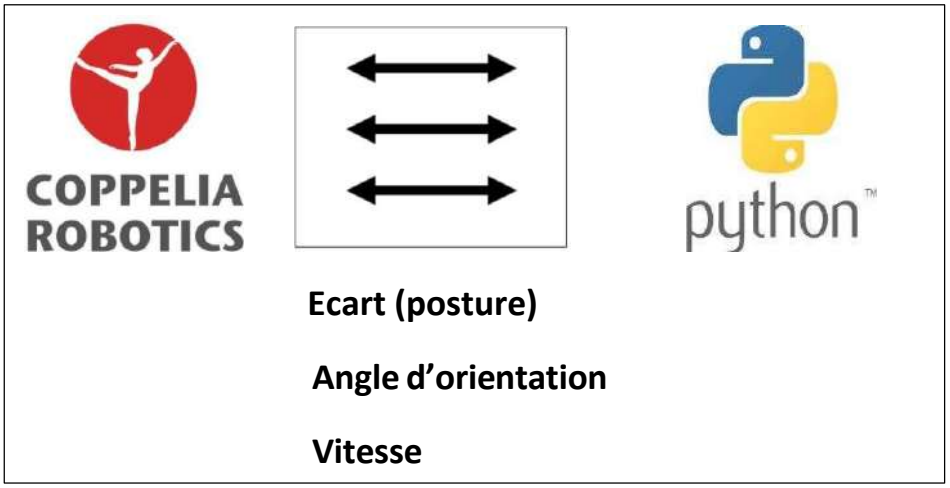


Figure 3.2.1 : contrôle des moteurs du robot à partir d'un programme python et tracer de la trajectoire rectiligne

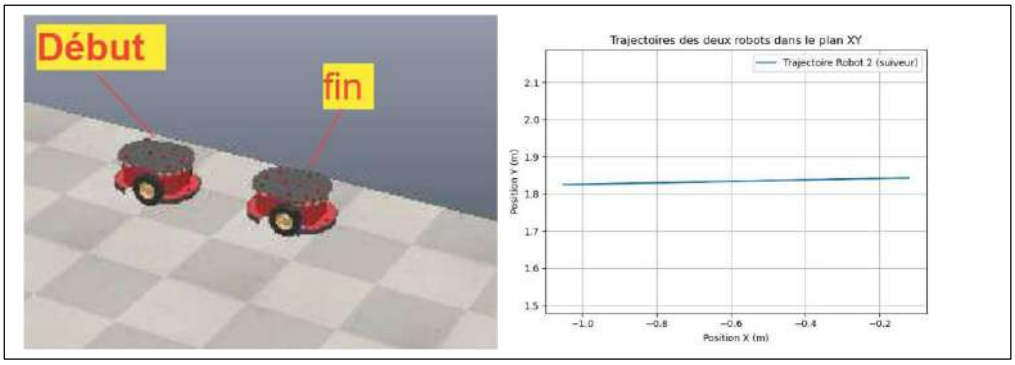


Figure 3.2.2 : contrôle des moteurs du robot à partir d'un programme python et tracer de la trajectoire rectiligne

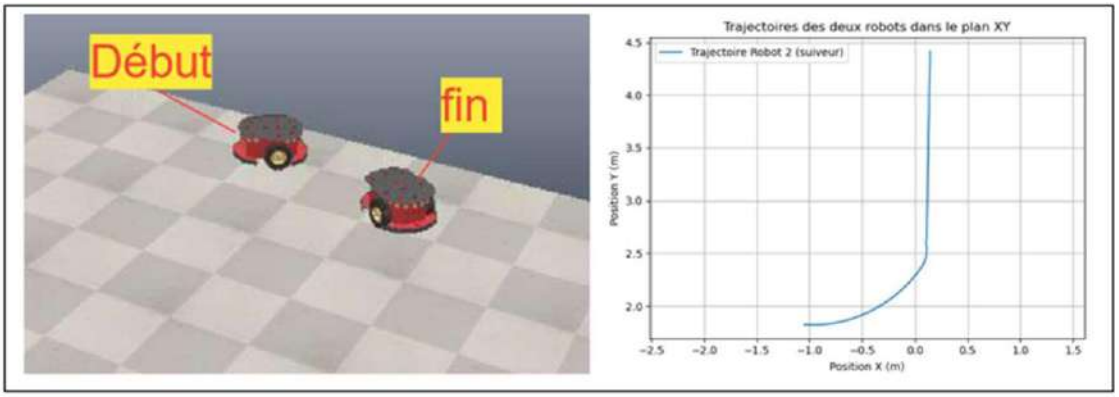


Figure 3.2.3: contrôle des moteurs du robot mobile par un programme python et tracer de la trajectoire circulaire

## 3. 3- Configuration de la scène

Avant de procéder à l'asservissement du robot suiveur, il est important au préalable d'effectuer une configuration de la scène.
Dans CoppeliaSim, les deux robots, suiveur et référent, sont des Pioneer 3DX. Ces derniers interagissent dans l'interface comme des objets dynamiques, donc ils peuvent être sujets à des comportements physiques comme des collisions. Pour résoudre ce problème, on paramètre le Mask de chaque roue des deux robots pour éviter les collisions. Donc, le Mask permet de déterminer quel type d'objet peut entrer en collision avec un autre objet du même type ou de type différent. La figure (3.4.1) suivante montre comment configurer le Mask des roues.

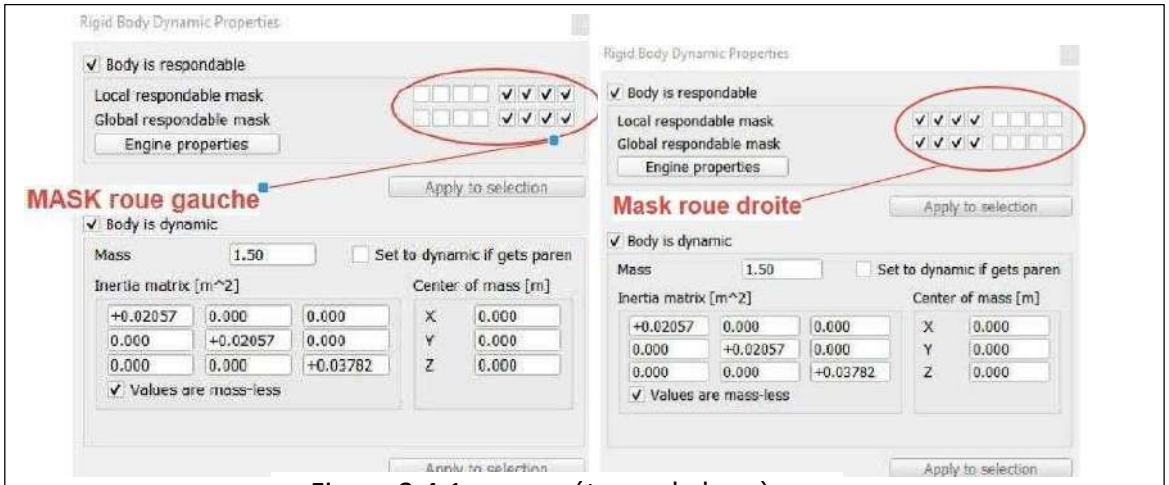


Figure 3.4.1 : paramétrage de la scène

## 4- DESCRIPTION DU PIONEER 3DX

Le Robot pioneer 3-DX est largement utilisé dans la recherche en robotique mobile, notamment pour la cartographie, la localisation, la télé-opération, la surveillance, la reconnaissance ou encore l'exploration autonome. Il est équipé de 16 capteurs à ultrasons (orientés vers l'arrière et vers l'avant) ; deux roues motrices principales à l'arrière, chacune entraînée par un moteur indépendant, ce qui permet un suivi précis de la position angulaire de chaque roue. Elles assurent la propulsion et la direction du robot par différentiel de vitesse : la différence de vitesse entre les deux roues permet au robot de tourner ou de se déplacer en ligne droite. La figure 3.3.1 montre la structure du robot mobile Pioneer 3DX.

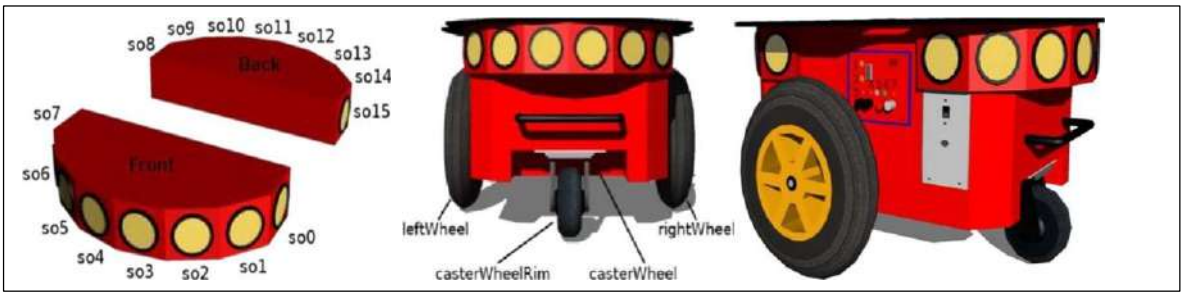


Figure 4 : robot mobile Pioneer 3DX

# 5 - Première approche : commande proportionnelle

## 5.1 - Introduction à la commande proportionnelle et au modèle cinématique

Dans la figure 5.1, on observe le modèle cinématique du robot Pioneer 3DX. Ce modèle illustre la configuration du robot dans le plan, en intégrant plusieurs facteurs importants. Le paramètre $v$ représente la vitesse linéaire du robot, c'est-à-dire la vitesse à laquelle le robot se déplace dans la direction de son orientation. L'angle θ représente l'orientation du robot par rapport à un repère global (par rapport à l'axe x).

Les roues motrices situées à l'arrière du robot, peuvent tourner de manière indépendante.

On a :

- $\Phi R$ : la vitesse angulaire de la roue droite

- $\Phi L$ : la vitesse angulaire de la roue gauche
- L : la distance entre les deux roues
- $\omega$ : la vitesse angulaire du robot autour de son axe vertical ;

La distance L est un paramètre essentiel dans le calcul de la rotation du robot, car elle influe directement sur la relation entre les vitesses angulaires des roues et la rotation du robot lui-même.

Ce modèle est à la base de la commande du robot, car il relie les vitesses des roues aux mouvements globaux du robot dans l'espace.

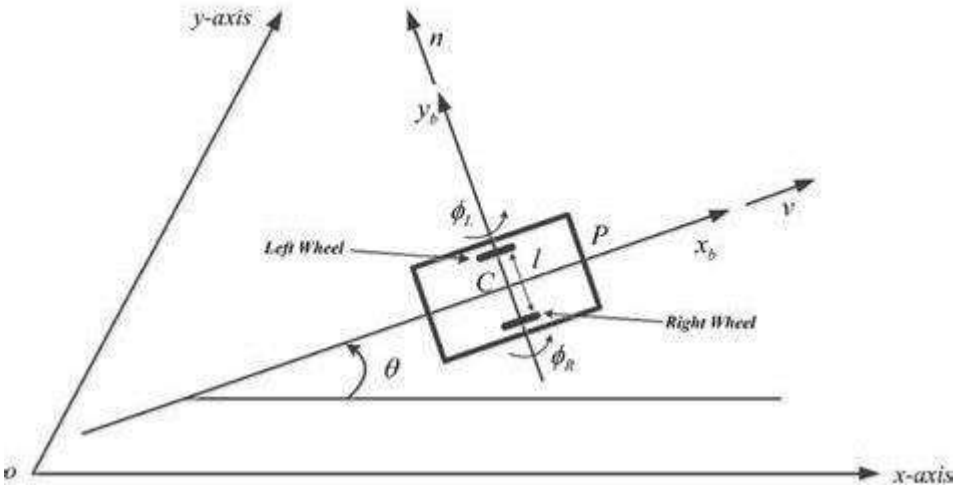


Figure 5.1 : Modèle cinématique du robot Pioneer 3DX

La configuration du robot à un instant t est décrite par le vecteur :

$$q(t) = \begin{bmatrix} x(t) \\ y(t) \\ \theta(t) \end{bmatrix} \qquad \text{(eq 5.1)}$$

où :

- x(t), y(t) sont les coordonnées du robot dans le repère global,
- Θ (t) est l'orientation du robot par rapport à l'axe x du repère.

La dynamique de mouvement est donnée par :

$$\dot{q} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\Theta} \end{bmatrix} = \begin{bmatrix} \cos(\Theta) & 0 \\ \sin(\Theta) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix} \qquad \text{(eq 5.2)}$$

où :

- $v$ est la vitesse linéaire du robot,
- $\omega$ est la vitesse angulaire (taux de rotation autour de son axe vertical).

Les vitesses linéaires des roues gauche et droite peuvent être exprimées à partir de v et ω :

$$v_D = v + a\omega \quad \text{et} \quad v_G = v - a\omega \qquad \text{(eq 5.3)}$$

où :

- $a=\frac{L}{2}$ ; représente la moitié de la distance entre les deux roues motrices (ou la distance du centre du robot à chaque roue).

Ces équations permettent de convertir les commandes globales du robot $(v, \omega)$ en commandes spécifiques aux roues droite et gauche $v_D$ et $v_G$ . Ces derniers sont effectivement envoyés aux moteurs pour piloter le déplacement du robot.

## 5.2 - Principe du suivi suiveur-référence

Dans cette configuration, le robot de référence suit une trajectoire prédéfinie, et le robot suiveur doit ajuster sa position pour maintenir une formation relative. En se basant sur le modèle cinématique précédent, une loi de commande est établie afin que le robot suiveur corrige ses vitesses linéaire et angulaire pour réduire l'écart avec le robot de référence, selon la formation définie par le vecteur d'écart $F_d = [F_{dx}, F_{dy}] = (x_r - y_r, x - y)^T$ où $(x_r, y_r)$ et $(x, y)$ représentent respectivement les coordonnées du leader, suiveur.

Les robots adoptent une stratégie de contrôle de type suiveur- référence, où seul le déplacement du robot de référence est commandé. Le robot suiveur, de son côté, modifie sa trajectoire en temps réel sur la base de l'état relatif.

Ce système est simulé dans le cadre de CoppeliaSim ,où les robots interagissent et adaptent leurs comportements afin de maintenir une formation stable.

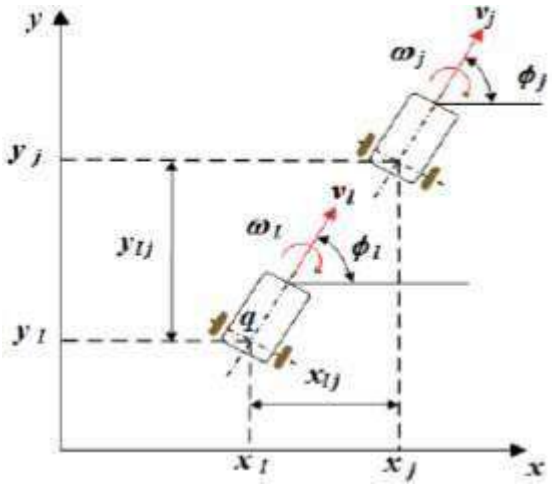


Figure 5.2 : Configuration du robot référence et le robot Leader

Le suivi repose sur une commande cinématique basée sur l'estimation des erreurs de position et d'orientation.

Soient $P_1 = (x_1, y_1)$ la position du robot leader et $P_2 = (x_2, y_2)$,celle du suiveur. L'erreur de position est caractérisée comme le vecteur qui joint ces deux points :

$$\Delta x = x1 - x2 \; ; \; \Delta y = y1 - y2 \qquad \text{(eq 5.4)}$$

On définit alors la distance entre les deux robots, notée $d$ :

$$d = \sqrt{(\Delta x)^2 + (\Delta y)^2} \qquad \text{(eq 5.5)}$$

On utilise cette distance pour déterminer la commande en vitesse linéaire. Par ailleurs, l'orientation actuelle du robot suiveur est notée $\theta$, tandis que l'angle cible α, c'est-à-dire la différence d'angle entre sa position et celle de la référence, est déterminé par :

$$\alpha = arctan\ 2(\Delta y, \Delta x) \qquad \text{(eq 5.6)}$$

L'erreur angulaire est définie comme la différence entre l'angle visé et l'orientation réelle du robot suiveur :

$$\theta_{\text{err}} = \alpha - \theta \qquad \text{(eq 5.7)}$$

Elle est normalisée dans la gamme $[-\pi, \pi]$ pour éviter les discontinuités lors du passage de $\pi$ à $-\pi$.

## 5.3 - Loi de commande proportionnelle

On applique alors une loi de commande proportionnel, où la vitesse linéaire $v$ et la vitesse angulaire $\omega$ sont déterminées de la manière suivante :

$$v = K_v . d \text{ avec } v \leq v_{max} \qquad \text{(eq 5.8)}$$

$$w = k_\omega . \theta_{err} \qquad \text{(eq 5.8)}$$

Où $K_v$ et $K_\omega$ sont des gains de commande positifs. Dans notre implémentation, les valeurs suivantes ont été utilisées,$K_v$ = 28 et $K_\omega$ = 2.5 , avec une vitesse maximale $v_{max}$ =10 $rad/s$ .

Les paramètres clés dans la loi de commande sont les gains de commande. Ils définissent la capacité de réaction du robot suiveur aux erreurs de distance et d'orientation. Le gain linéaire $Kv$ a un impact sur la rapidité à laquelle le robot diminue l'écart qui le sépare du robot de référence. Un gain trop faible conduit à un comportement lent et peu réactif, tandis qu'un gain trop élevé peut provoquer des oscillations ou une instabilité, surtout lorsque le robot est proche du référence. Le gain angulaire $K_\omega$ définit la vitesse à laquelle le robot ajuste son orientation pour viser l'objectif. Un gain trop bas empêche le robot de bien s'orienter vers le leader, tandis qu'un gain excessif des mouvements brusques et des trajectoires irrégulières.

Les vitesses $v$ et $\omega$ sont ensuite converties en vitesses de roues ($v_G$et $v_D$)(eq 5.3), conformément au modèle cinématique précisé précédemment, puis transmises aux moteurs du robot suiveur via l'interface de commande de CoppeliaSim.

Cette loi de commande est mise en œuvre de façon itérative à chaque étape de simulation, avec une fréquence d'échantillonnage de 100 $Hz$ .Les données de position,

d'orientation et les erreurs sont enregistrées afin de créer des diagrammes qui facilitent l'évaluation des performances du suivi.

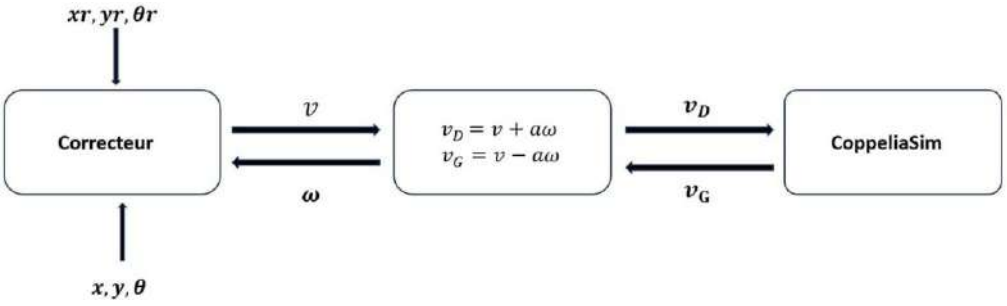


Figure 5.3 : schéma synthétique représentant les différentes variables et les relations fonctionnelles

## 5.4 - Description de la méthode du triangle

La méthode du triangle constitue la base géométrique utilisée pour le calcul des erreurs nécessaires à l'application de la loi de commande proportionnelle décrite en section 5.3. Cette méthode facilite la commande en la réduisant à deux paramètres fondamentaux : l'écart entre les deux robots (l'un des côtés du triangle) et l'angle formé entre leurs orientations (angle au sommet du triangle).

L'erreur angulaire est donc définie non pas en fonction de l'orientation exacte du robot de référence, mais en se basant sur la direction du segment qui joint les deux robots. Par conséquent, l'orientation du robot de référence n'est pas utilisée directement. Cela nous permet d'exprimer la correction d'orientation du suiveur par rapport à la ligne qui relie les deux robots, en fonction de leur configuration relative.

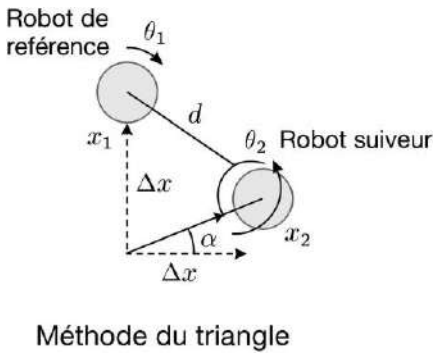


Figure 5.4 : La méthode de triangle

## 5.5 - Résultats des simulations

Des simulations ont été réalisées afin d'évaluer le comportement du robot suiveur dans un scénario de suivi fondé sur une commande proportionnelle. Dans l'environnement simulé

en 2D, le robot de référence suit une trajectoire imposée, tandis que le robot suiveur ajuste sa vitesse linéaire et angulaire pour maintenir une configuration relative stable.

Les trajectoires présentées sur les diagrammes démontrent une inclinaison du robot suiveur à converger vers le trajet suivi par le robot de référence, comme en témoigne la superposition presque exacte des courbes de trajectoire (figure 5.5). Les flèches dénotent l'orientation du robot suiveur à différents moments, illustrant son ajustement constant face à l'erreur de direction, ainsi qu'un rapprochement graduel des positions $(x, y)$ au cours du temps.

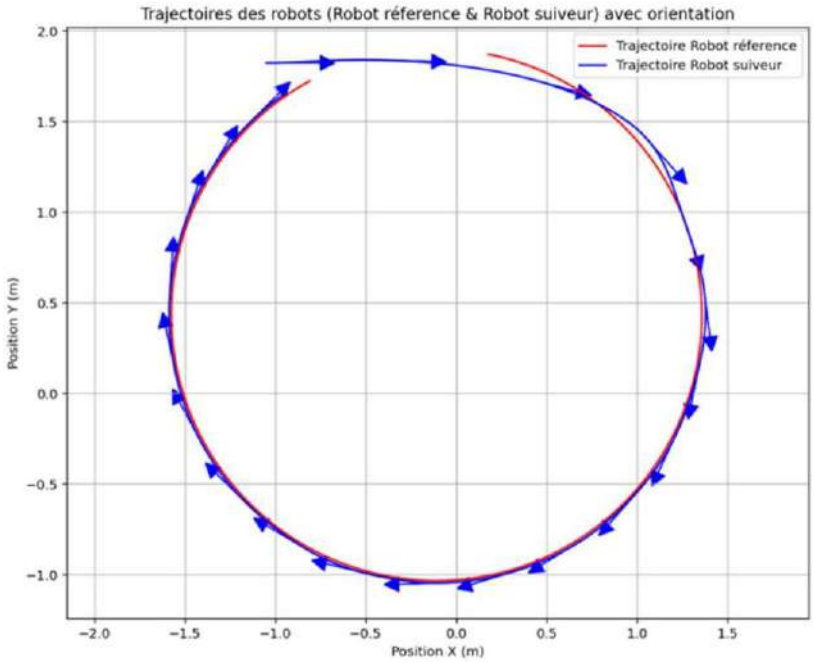


Figure 5.5: Trajectoires des robots avec orientation

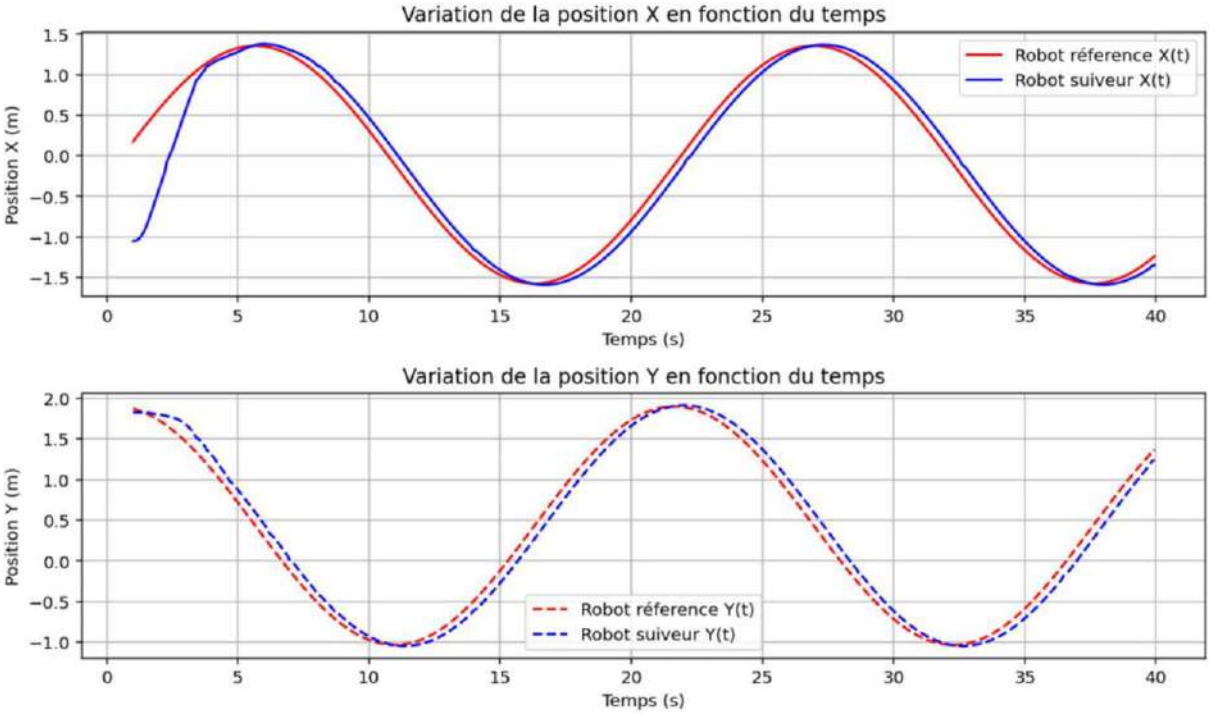


Figure 5.6: La variation de position $(x, y)$ des 2 robots

L'analyse de la variation des erreurs de distance et d'angle au fil du temps révèle une réduction notable de ces dernières, révélant ainsi la fiabilité et de l'efficacité du contrôleur proportionnel utilisé. L'écart de distance se réduit progressivement, illustrant un mouvement progressif du robot suiveur vers le robot de référence. L'erreur angulaire se dirige aussi vers des valeurs plus basses, montrant une correction de l'orientation.

Bien que la loi de commande offre un suivi global satisfaisant, il est à noter que les erreurs de position et d'orientation ne tendent pas à converger parfaitement vers zéro. Effectivement, l'erreur d'orientation fluctue autour d'une valeur moyenne qui n'est pas nulle, tandis que l'erreur de position reste presque stable à une certaine valeur positive. Ces phénomènes peuvent s'expliquer par plusieurs facteurs liés à la modélisation et à la commande utilisée.

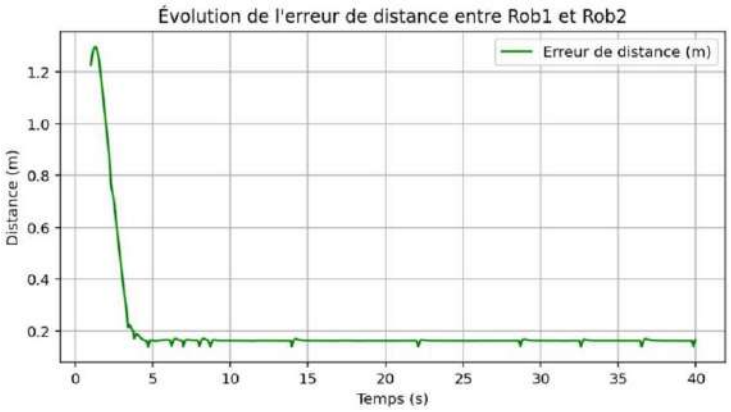


Figure 5.7: Evolution de l'erreur de distance

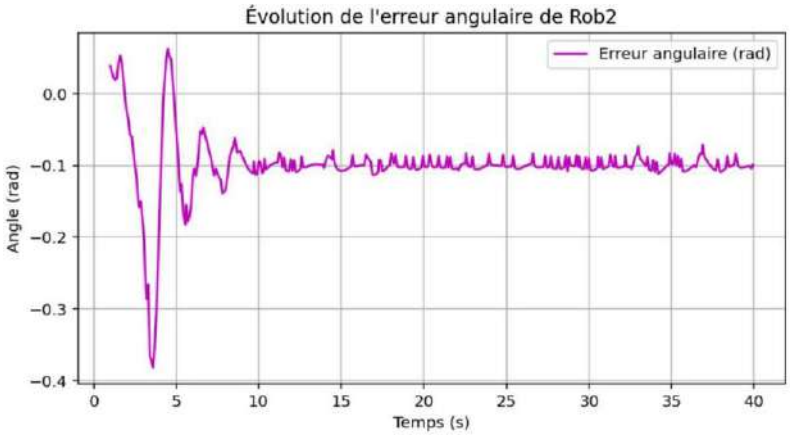


Figure 5.8 : Evolution de l'erreur angulaire

Une série de tests a été effectuée en faisant augmenter les valeurs des gains proportionnels $Kv$ et $K\omega$. Nous avons remarqué, via les diagrammes (figure 5.9), que l'erreur commence à se réduire et à tendre vers une valeur proche de zéro. Toutefois, à mesure que cette convergence débute, le système commence à osciller de manière marquée. Ces variations importantes, qui se produisent au moment où l'erreur tend vers zéro, mettent en évidence une instabilité du système. Ceci met en évidence l'importance d'ajuster correctement les gains pour éviter un comportement trop agressif qui pourrait mettre en risque la stabilité générale du suivi.

Ce phénomène est également visible dans la trajectoire du robot suiveur (figure 5.10), qui dépasse la position du robot de référence avant d'osciller autour de sa trajectoire. Cette instabilité est attribuée à une commande proportionnelle trop réactive, principalement en raison d'un gain linéaire $Kv$ élevé. À chaque fois qu'il y a un dépassement, le robot ajuste de façon excessive dans le sens contraire, ce qui intensifie les oscillations.

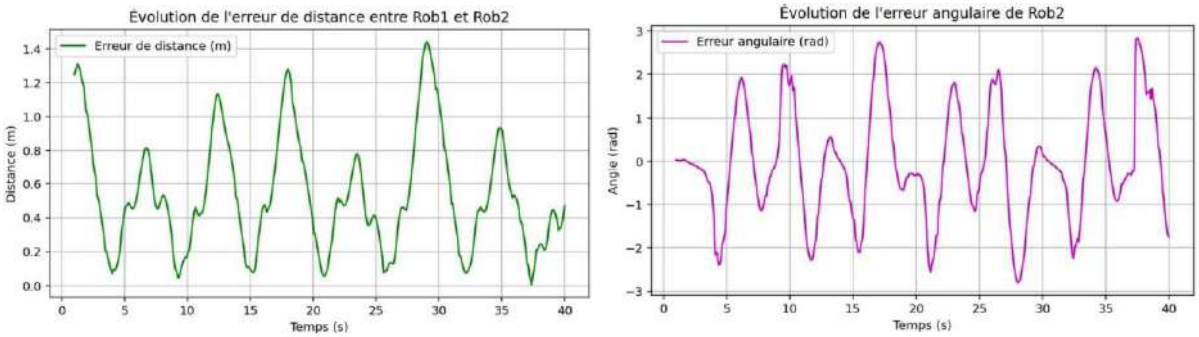


Figure 5.9: Les erreurs angulaire et de distance avec des gains élevés

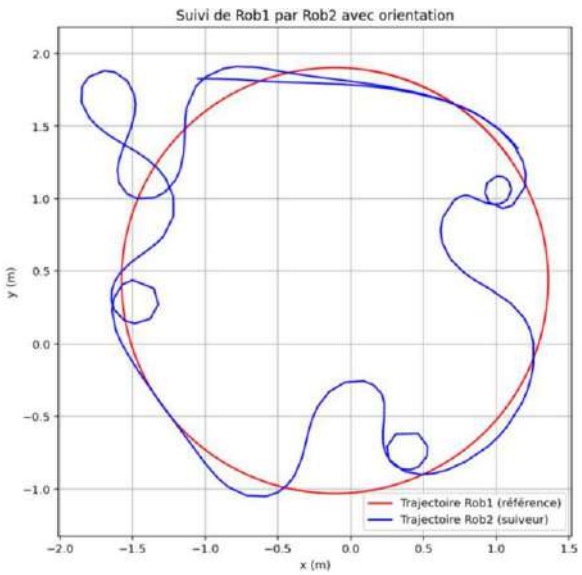


Figure 5.10: Trajectoires des robots avec orientation avec des gains élevés

Finalement, l'approche du triangle offre le bénéfice d'une application facile et d'une bonne flexibilité dans des situations où il est difficile de déterminer avec précision l'orientation absolue du robot principal. Cette approche, qui utilise une référence d'orientation relative, entraîne naturellement des fluctuations dans l'erreur angulaire, ce qui limite la convergence parfaite. Ces oscillations sont donc une conséquence directe du compromis choisi entre simplicité et précision.

Pour améliorer la précision du suivi et corriger les marges d'erreur observées, nous allons dans la suite utiliser une loi de stabilisation avec orientation. Cette commande agit non seulement sur l'orientation du robot suiveur, mais prend également en compte l'orientation du robot de référence. Cette méthode permet une meilleure gestion de l'erreur angulaire, minimisant ainsi les oscillations et garantissant une convergence plus stable et plus rapide vers le parcours souhaité.

# 6. Commande par retour d'état sur l'erreur de posture

## 6.1. Principe de la commande

Prenant en compte les problèmes rencontrés lors de notre première tentative de commande, nous avons cherché une méthode alternative pour piloter le robot suiveur, de manière à contourner les difficultés observées.

En nous appuyant sur les travaux de [Bayle ,2016], nous avons tout d'abord tenté de définir l'erreur de posture entre les deux robots dans le repère du robot de référence, afin de rendre cette erreur indépendante de la position absolue dans l'espace. Nous considérons alors la formule établie par Bayle (2016) :

$$\begin{bmatrix} x_e \\ y_e \\ \theta_e \end{bmatrix} = \begin{bmatrix} \cos\theta_r & \sin\theta_r & 0 \\ -\sin\theta_r & \cos\theta_r & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x - x_r \\ y - y_r \\ \theta - \theta_r \end{bmatrix} \qquad \text{eq. 6.1}$$

De cette manière, nous parvenons à projeter l'erreur dans le repère du robot suiveur.

En dérivant cette erreur, nous obtenons son évolution en fonction des vitesses v, ω (vitesses linéaires et angulaires du robot suiveur) et $v_r$, $\omega_r$ (vitesses linéaires et angulaires du robot de référence). À partir de cette expression, nous effectuons un changement de variables:

- $x_1 = x_e$ eq. 6.2
- $x_2 = y_e$ eq. 6.3
- $x_3 = \tan\theta_e$ eq. 6.4
- $u_1 = v\cos\theta_e - v_r$ eq. 6.5
- $u_2 = \frac{\omega - \omega_r}{\cos(\theta_e)^2}$ eq. 6.6

Ce changement de variable transforme le système dynamique non linéaire — obtenu après la dérivation de l'erreur — en un système dynamique linéaire, avec lequel on peut travailler de manière classique. Nous avons ensuite appliqué la loi de commande par retour d'état proposée par Bayle (2016), En établissant une relation entre les variables $x_1$, $x_2$, $x_3$, $u_1$, $u_2$, les constantes strictement positives $k_1$, $k_2$, $k_3$ et la vitesse linéaire $v_r$ , nous obtenons:

- $u_1 = -k_1 \mid v_r \mid (x_1 + x_2 x_3)$ eq. 6.7
- $u_2 = -k_2 v_r x_2 - k_3 |v_r| x_3$ eq. 6.8

## 6.2. Application de la commande sous CoppeliaSim

Une fois la loi de commande établie, nous sommes passés à l'implémentation sur CoppeliaSim en utilisant le langage Python. L'objectif était de développer un script capable de piloter le robot suiveur en temps réel.

Dans un premier temps, nous avons utilisé les formules issues de la loi de commande pour déterminer les vitesses linéaires et angulaires à appliquer au robot suiveur. Cela nous a permis d'aboutir aux équations suivantes (eq. 6.9 et eq. 6.10) :

- $v = \frac{(u_1 + v_r)}{\cos\theta_e}$ eq. 6.9
- $\omega = u_2 (\cos\theta_e)^2 + \omega_r$ eq. 6.10

À partir des équations obtenues, nous avons pu identifier les données nécessaires au calcul des vitesses à appliquer.

## 6.3. Les Données

Depuis la scène de CoppeliaSim, nous avons récupéré la position des deux robots dans le repère global de la scène, sous la forme de coordonnées (x,y,z).

Nous avons également extrait l'orientation de chaque robot. Pour cela, nous avons développé une fonction permettant de ne récupérer que l'angle de rotation autour de l'axe Z . Nous avons aussi collecté les vitesses linéaire ($v_r$) et angulaire ($\omega_r$) du robot de référence.

Outre les données récupérées depuis CoppeliaSim, nous avons également calculé, dans notre code, les erreurs de position $X_e$ et $Y_e$ selon les axes X et Y ainsi que l'erreur angulaire ($\theta_e$) entre les deux robots.

À partir de ces données et de la loi de commande, nous avons calculé les vitesses linéaire et angulaire du robot suiveur .En suivant le même principe que pour le script de commande précédent, nous avons déterminé les vitesses nécessaires pour les roues droite et gauche, puis transmis ces valeurs à CoppeliaSim afin de contrôler le robot.

## 6.4. Simulation et Resultats

Pour les simulations, nous avons choisi d'utiliser la même trajectoire pour le robot de référence que celle de la commande proportionnelle, afin de pouvoir comparer les résultats. Dans un premier temps, nous avons expérimenté différents valeurs des constantes K1, K2 et K3 pour observer la réponse du système.

En fixant K1 dans l'ordre des dizaines, nous avons constaté une erreur statique sur la position, qui n'a pas été corrigée durant la durée de la simulation. Cette erreur est visible sur le graphique suivant (Figure 6.1), qui compare les positions en x et y des deux robots. Pour cette simulation, nous avons utilisé les paramètres suivants :

K1=10 K2=7.5 K3=2.5

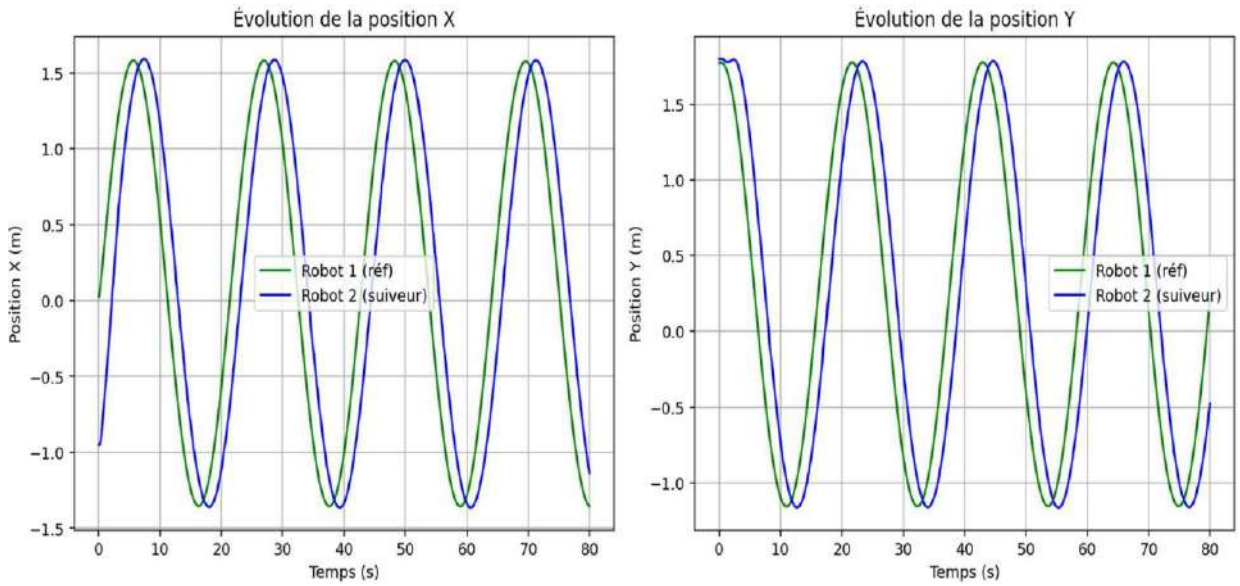


Figure 6.1 : Évolution des positions selon les axes x et y au cours du temps.

En tenant compte de ces observations, nous avons décidé d'augmenter la constante K1 afin de réduire au maximum cette erreur statique. Cependant, en augmentant K1 , la vitesse du robot suiveur à distance du robot de référence devenait trop élevée, et dû à l'inertie du mouvement, le système devenait instable. Pour pallier ce problème, nous avons introduit une limite maximale sur la vitesse du robot suiveur, ce qui a permis de stabiliser le système. La Figure 6.2 illustre la trajectoire des deux robots sans l'application de cette limitation de vitesse maximale :

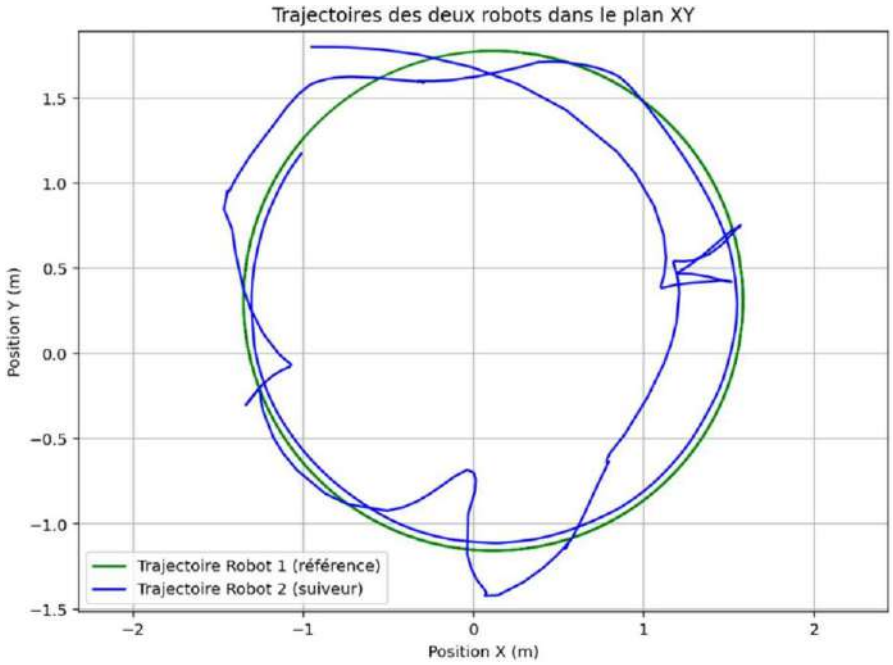


Figure 6.2 : Trajectoires des deux robots sans vitesse maximale.

Nous avons posé $V_{Max} = 7$ et avec cette modification, nous avons augmenté K1 à la valeur K1=140, tout en fixant K2=7.5 et K3=2.5.

Cette configuration a permis de réduire l'erreur de position à une valeur très proche de zéro, tout en maintenant la stabilité du système dans toutes les situations.

Les résultats obtenus lors de la simulation avec ces paramètres sont présentés ci-dessous (Figure 6.3):

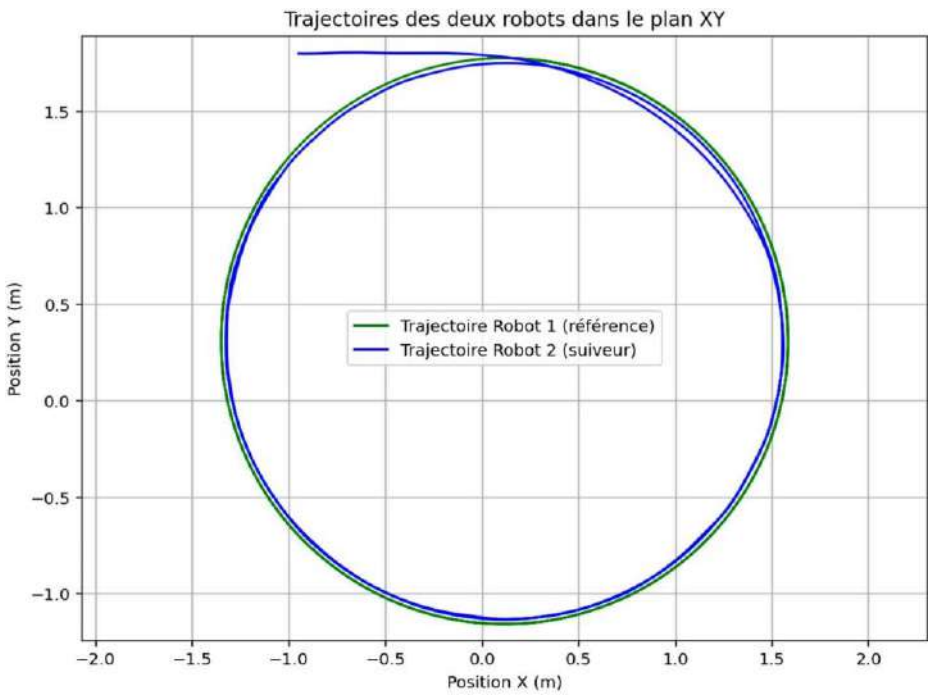


Figure 6.3 : Trajectoires des deux robots avec vitesse maximale.

La Figure 6.3 illustre les trajectoires des deux robots : en vert, celle du robot de référence, et en bleu, celle du robot suiveur. On peut observer une légère erreur entre les deux trajectoires, que nous détaillons dans les graphiques suivants (Figure 6.4) :

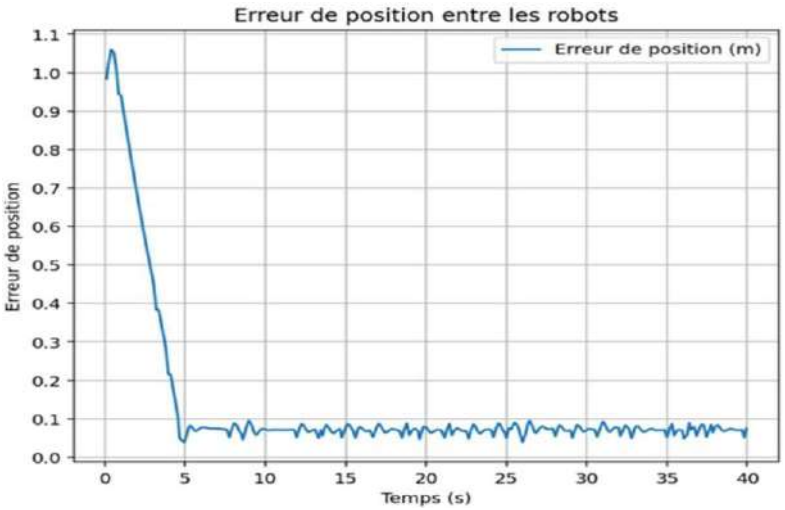


Figure 6.4 : Erreur de position entre les robots.

Cette figure présente le graphe de l'erreur de position. On y observe que le robot suiveur met environ 5 secondes pour rattraper le robot de référence, puis maintient sa trajectoire avec une erreur oscillant autour de 0,08 mètres. Le graphe suivant (Figure 6.5) illustre quant à lui l'évolution de l'erreur d'orientation entre les deux robots :

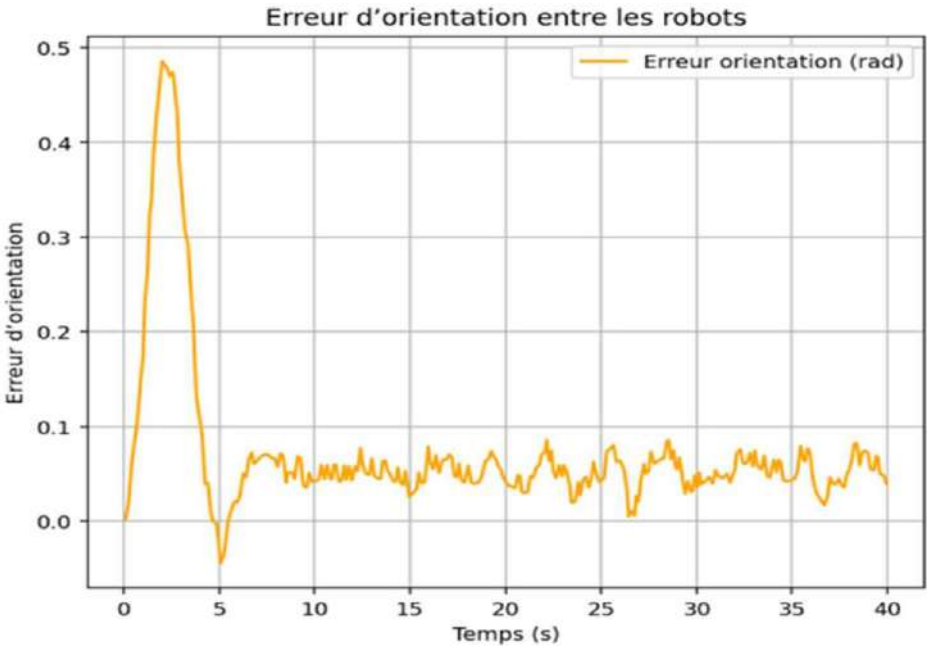


Figure 6.5 : Erreur de orientation entre les robots.

Nous avons également récupéré l'évolution des positions des deux robots selon les axes x et y au cours du temps (Figure 6.6) :

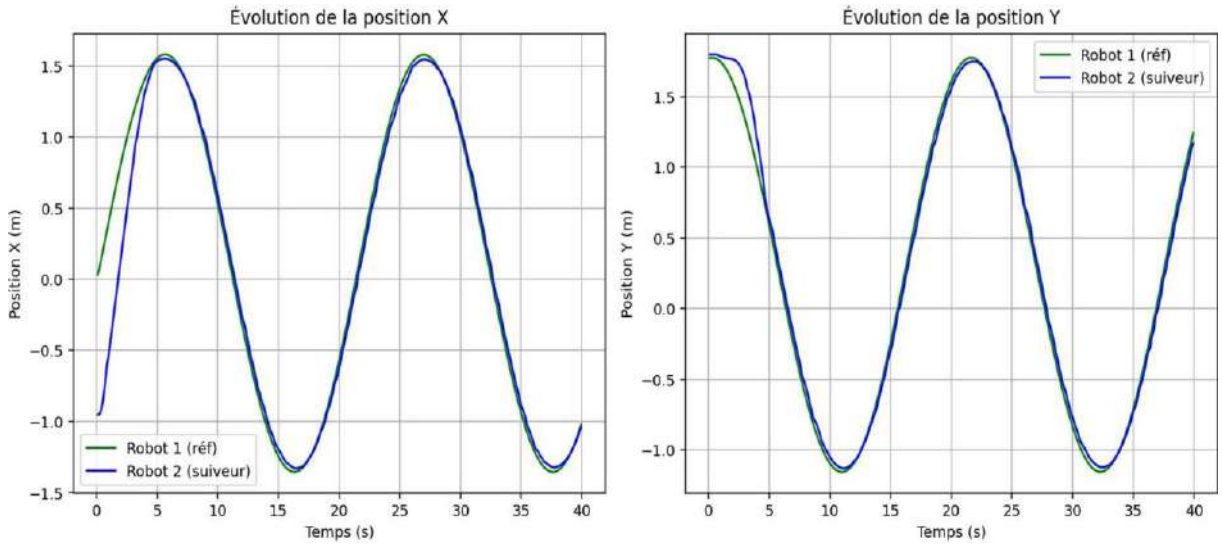


Figure 6.6 : Évolution des positions selon les axes x et y au cours du temps.

Ces graphiques (Figure 6.6) montrent qu'avec cette commande, il n'y a pas de décalage de position au cours du temps, contrairement à la commande proportionnelle. C'est à ce stade que nous avons procédé à une comparaison des résultats obtenus avec les deux lois de commande, ainsi qu'à des tests dans des situations un peu plus complexes.

## 6.5. Comparaison

Pour cette étape nous avons imposé des virages brusques du robot de référence, dans le but de tester la capacité de la commande à maintenir la trajectoire.

Nous avons également simulé cette même trajectoire en utilisant la commande proportionnelle, afin de comparer les performances des deux méthodes.

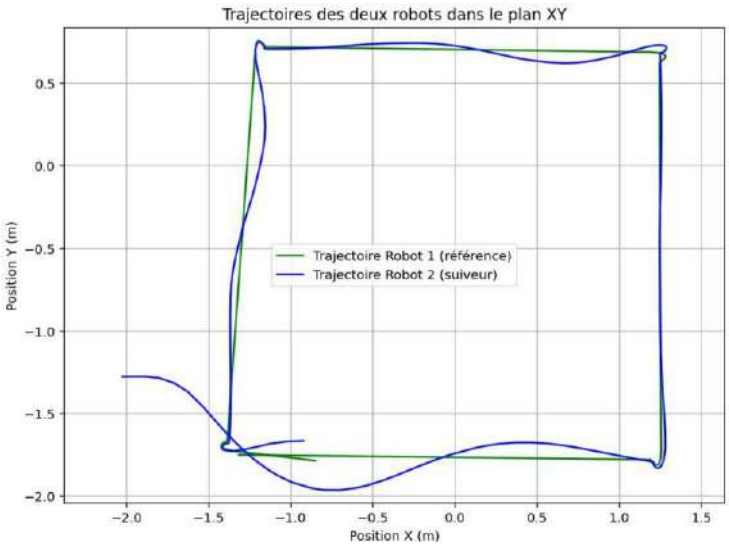


Figure 6.7 : Trajectoires des deux robots pour la nouvelle trajectoire

On peut observer sur la Figure 6.7 que le robot suiveur parvient à stabiliser sa trajectoire, bien que de manière plus difficile que précédemment. En revanche, avec la commande proportionnelle, en utilisant les mêmes valeurs des constantes Kv et Kw que dans la section 5.3, la trajectoire obtenue est celle de la Figure 6.8:

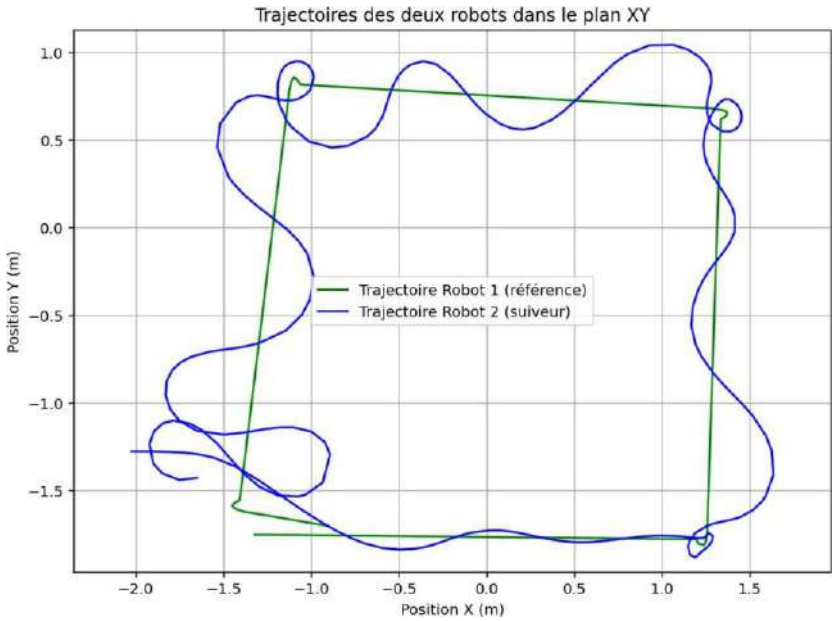


Figure 6.8 : Trajectoires – commande proportionnelle Kv=28

En observant clairement la Figure 6.8, on constate que le système est instable. Nous avons alors réduit la constante Kv=20 ce qui a permis d'obtenir un système stable, comme illustré dans la Figure 6.9:

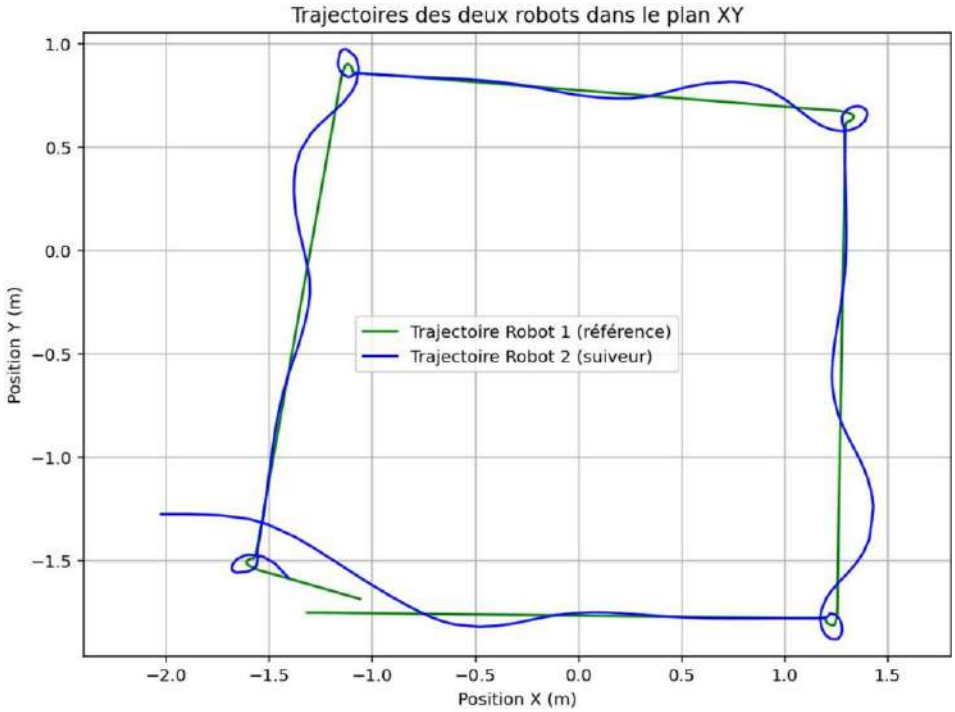

Figure 6.9 : Trajectoires – commande proportionnelle Kv=20

Maintenant que la commande proportionnelle est stabilisée, passons à la comparaison des performances entre les deux lois de commande.

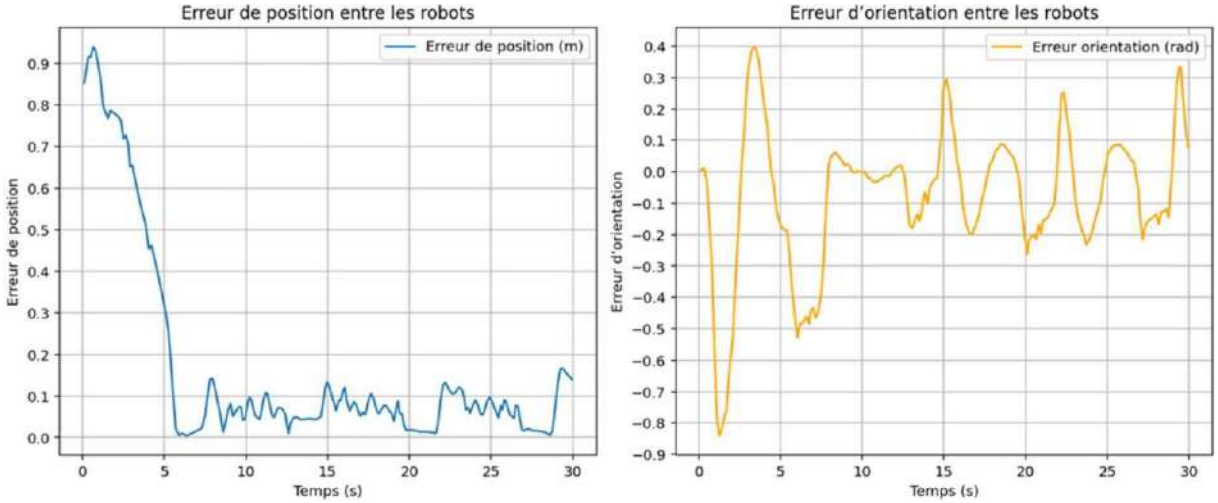


Figure 6.10 : Erreur de position et orientation – commande de Bayle (2016)

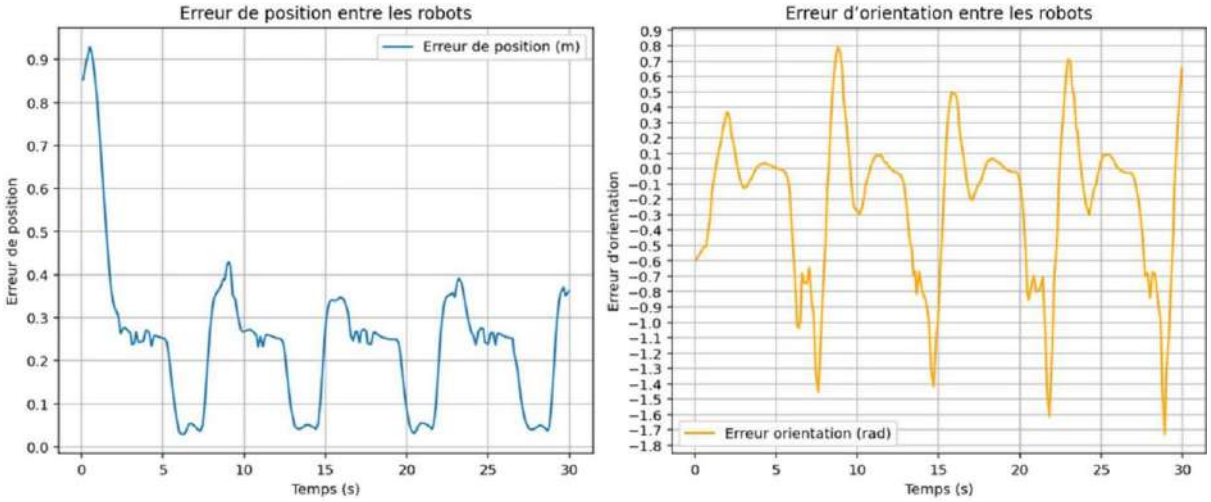


Figure 6.11 : Erreur de position et orientation – commande proportionnelle

En observant ces quatre graphiques, on constate que pour cette trajectoire plus difficile à suivre, la commande proposée par Bayle (2016) (Figure 6.10) maintient l'erreur de position autour de 0,1 mètre, tandis qu'avec la commande proportionnelle (Figure 6.11), l'erreur devient nettement plus importante. Il en est de même pour l'erreur angulaire.

En examinant maintenant l'évolution des positions selon les axes x et y au cours du temps, nous obtenons :

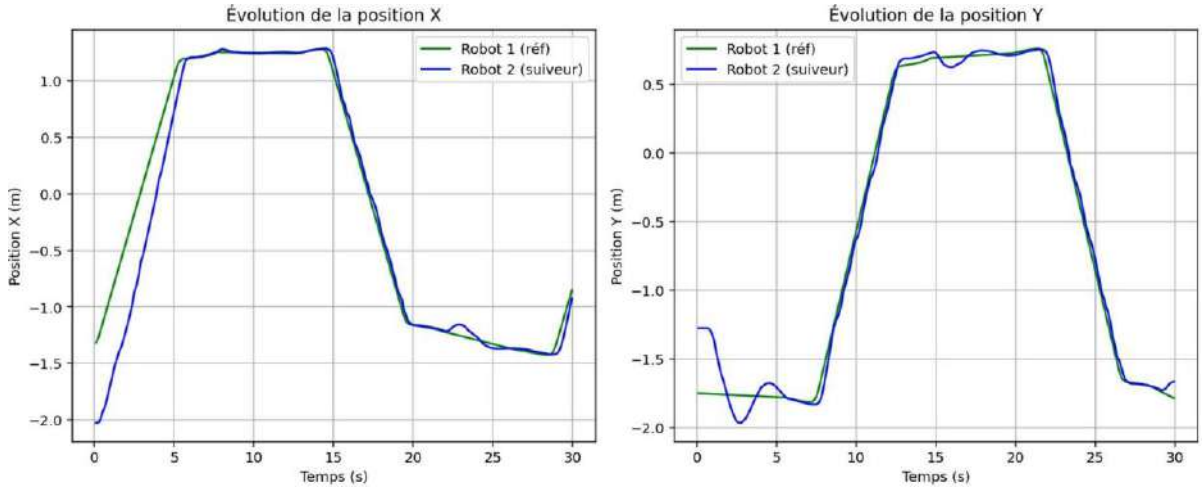


Figure 6.12 : Évolution des positions selon les axes x et y – commande de Bayle (2016)

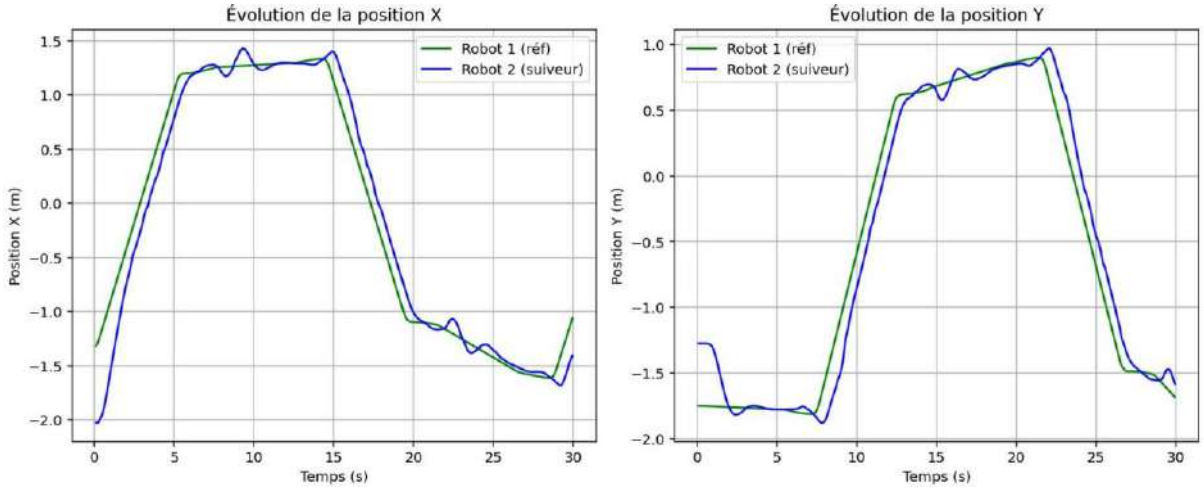


Figure 6.13 : Évolution des positions selon les axes x et y – commande proportionnelle

On observe ici le résultat attendu après l'ensemble de nos études : la commande proposée par Bayle (2016) (Figure 6.12) permet au robot suiveur de suivre fidèlement la trajectoire du robot de référence, avec une erreur de position minimale comme montré précédemment, et un décalage temporel très faible.

En revanche, la commande proportionnelle (Figure 6.13), pour rester stable, nécessite de maintenir un décalage temporel plus important.

# 7. Conclusion

Ce projet avait pour objectif de stabiliser le mouvement d'un robot à partir d'un robot de référence. L'enjeu principal était de proposer une loi de commande capable de commander le robot suiveur de façon à ce que sa trajectoire soit stable.

Au fil du développement, nous avons travaillé pour trouver premièrement les moyens de faire le lien entre le simulateur que nous avons utilisé (CoppeliaSim) et Python, ce qui nous a posé des problèmes d'ordre logiciel, étant donné la difficulté de travailler avec un logiciel nouveau.

Une fois cette étape franchie, nous avons pu établir et tester une loi de commande qui atteignait notre objectif. Dans des situations peu exigeantes, performaient acceptablement. Mais en essayant de mettre à l'épreuve notre travail dans des situations plus compliquées, nous avons constaté des limitations de stabilité liés à la propre loi de commande. Nous avons alors exploré une nouvelle méthode qui, bien qu'elle présentait aussi certaines problématiques — notamment liées à la complexité de la loi de commande — nous a permis d'obtenir des résultats plus précis et désirables.

Étant donné le fonctionnement de ces lois de commande, il serait intéressant par la suite de déterminer si leur application peut s'étendre à des robots différents du Pioneer P3-DX, en analysant leur efficacité sur des robots holonomes, par exemple.

Au-delà des aspects techniques (robots mobiles, logiciels de programmation et de simulation), ce projet nous a permis de développer des compétences transversales telles que la gestion de projet, la communication au sein de l'équipe, et la résolution de problèmes concrets dans des délais limités.

En somme, ce projet a constitué une expérience formatrice, à la fois sur le plan académique et personnel, en nous préparant aux exigences d'un travail collaboratif en conditions quasi-professionnelles.

# Annexe

## Annexe A – Code Python Loi de Commande proportionnel

```
import numpy as np
import time
import math
import matplotlib.pyplot as plt
from coppeliasim_zmqremoteapi_client import RemoteAPIClient


def get_orientation(sim, handle):
    # Retourne uniquement l'angle autour de Z (en radians)
    euler = sim.getObjectOrientation(handle, -1)
    return euler[2]


def get_position(sim, handle):
    # Retourne la position en (x,y,z)
    return np.array(sim.getObjectPosition(handle, -1))


# Connexion à CoppeliaSim
client = RemoteAPIClient()
sim = client.require('sim')


# Démarrage de la simulation (si elle n'est pas déjà en cours)
if sim.getSimulationState() == sim.simulation_stopped:
    sim.startSimulation()


# Donne un peu de temps pour que tout soit lancé
time.sleep(1)


# Récupération des handles des robots et moteurs
Rob2 = sim.getObject('/PioneerP3DX[0]')
Rob1 = sim.getObject('/PioneerP3DX[1]')

moteur_gauche2 = sim.getObject('/PioneerP3DX[0]/leftMotor')
moteur_droit2 = sim.getObject('/PioneerP3DX[0]/rightMotor')


# Temps de simulation
Temps = sim.getSimulationTime()
Temps_de_simulation = 40  # secondes


# Constantes
Kv = 28
Kw = 2.5
Vitesse_max = 10
L = 2  # demi-distance entre les roues
Interval_Echantillonage = 0.01


# Enregistrements pour visualisation
positions_rob1 = []
```

```
positions_rob2 = []
orientations_rob2 = []
erreurs_distance = []
erreurs_angulaires = []
temps_simulation = []


try:
    while Temps < Temps_de_simulation:
        # Récupération des positions et orientation
        Rob1_Pos = get_position(sim, Rob1)
        Rob2_Pos = get_position(sim, Rob2)
        Rob2_Orientation = get_orientation(sim, Rob2)


        # Calcul des erreurs
        dx = Rob1_Pos[0] - Rob2_Pos[0]
        dy = Rob1_Pos[1] - Rob2_Pos[1]
        distance = math.hypot(dx, dy)

        Angle = math.atan2(dy, dx)
        Erreur_Angulaire = (Angle - Rob2_Orientation + math.pi) % (2 * math.pi) - math.pi


        # Calcul des vitesses
        Vitesse_Lineaire = min(Vitesse_max, Kv * distance)
        Vitesse_Angulaire = Kw * Erreur_Angulaire

        Vitesse_Gauche = Vitesse_Lineaire - L * Vitesse_Angulaire
        Vitesse_Droite = Vitesse_Lineaire + L * Vitesse_Angulaire

        sim.setJointTargetVelocity(moteur_droit2, Vitesse_Droite)
        sim.setJointTargetVelocity(moteur_gauche2, Vitesse_Gauche)

        # Sauvegarde des données pour les graphiques
        positions_rob1.append(Rob1_Pos[:2])
        positions_rob2.append(Rob2_Pos[:2])
        orientations_rob2.append(Rob2_Orientation)
        erreurs_distance.append(distance)
        erreurs_angulaires.append(Erreur_Angulaire)
        temps_simulation.append(Temps)

        time.sleep(Interval_Echantillonage)
        Temps = sim.getSimulationTime()

    # Arrêt des robots
    sim.setJointTargetVelocity(moteur_droit2, 0)
    sim.setJointTargetVelocity(moteur_gauche2, 0)
    sim.stopSimulation()
    print("Simulation terminée proprement.")


except KeyboardInterrupt:
    sim.setJointTargetVelocity(moteur_gauche2, 0)
    sim.setJointTargetVelocity(moteur_droit2, 0)
    sim.stopSimulation()
    print("Simulation interrompue.")
```

```
plt.legend()
plt.grid(True)
plt.axis('equal')
plt.title('Trajectoires des robots (Robot réference & Robot suiveur) avec orientation')
plt.xlabel('Position X (m)')
plt.ylabel('Position Y (m)')
plt.savefig("trajectoires_xy.png", dpi=300)
plt.show()
```

# Annexe B – Code Python Loi de Commande 2

```
import matplotlib.pyplot as plt
from matplotlib.ticker import MultipleLocator
import numpy as np
import time
import math
from coppeliasim_zmqremoteapi_client import RemoteAPIClient

def get_orientation(sim, handle):
    # Retourne uniquement l'angle autour de Z (en radians)
    euler = sim.getObjectOrientation(handle, -1)
    return euler[2]

def get_position(sim, handle):
    # Retourne la position en (x,y,z)
    return np.array(sim.getObjectPosition(handle, -1))

def normaliser_angle(angle):
    return (angle + np.pi) % (2 * np.pi) - np.pi


# Connexion à CoppeliaSim
client = RemoteAPIClient()
sim = client.require('sim')

# Démarrage de la simulation (si elle n'est pas déjà en cours)
if sim.getSimulationState() == sim.simulation_stopped:
    sim.startSimulation()

# Récupération des handles des moteurs
Rob1 = sim.getObject('/PioneerP3DX[1]')
Rob2 = sim.getObject('/PioneerP3DX[0]')

moteur_gauche2 = sim.getObject('/PioneerP3DX[0]/leftMotor')
moteur_droit2 = sim.getObject('/PioneerP3DX[0]/rightMotor')

moteur_gauche1 = sim.getObject('/PioneerP3DX[1]/leftMotor')
moteur_droit1 = sim.getObject('/PioneerP3DX[1]/rightMotor')

#temps de simulation
Temps=sim.getSimulationTime()
Temps_de_simulation = 30

#Constantes
```

```
K1 = 140
K2 = 7.5
K3 = 2.5
Vitesse_max = 7
L = 2
Interval_Echantillonage = 0.01




#Debut de mouvement
try:
    while Temps < Temps_de_simulation:

        #Positions
        Rob1_Pos = get_position(sim, Rob1)
        Rob2_Pos = get_position(sim, Rob2)
        Theta_2 = get_orientation(sim, Rob2)
        Theta_1 = get_orientation(sim, Rob1)

        Vitesse_lineaire_rob1, Vitesse_angulaire_rob1 = sim.getObjectVelocity(Rob1)

        # Extraire la vitesse linéaire dans le plan XY
        vx, vy, _ = Vitesse_lineaire_rob1
        v_r = np.sqrt(vx**2 + vy**2)

        # Extraire la vitesse angulaire autour de Z (rotation dans le plan 2D)
        _, _, w_r = Vitesse_angulaire_rob1


        dx = Rob2_Pos[0] - Rob1_Pos[0]
        dy = Rob2_Pos[1] - Rob1_Pos[1]
        Erreur_Angulaire = normaliser_angle(Theta_2 - Theta_1)

        cos_Theta_1 = np.cos(Theta_1)
        sin_Theta_1 = np.sin(Theta_1)
        cos_Erreur_Angulaire = np.cos(Erreur_Angulaire)
        x_e = cos_Theta_1 * dx + sin_Theta_1 * dy
        y_e = -sin_Theta_1 * dx + cos_Theta_1 * dy

       #Changement de variable
        x1 = x_e
        x2 = y_e
        x3 = np.tan(Erreur_Angulaire)
        u1 = -K1 * abs(v_r) * (x1 + x2 * x3)
        u2 = (-K2 * v_r * x2) - (K3 * abs(v_r) * x3)

        #Vitesses
        Vitesse_Lineaire = (u1 + v_r)/cos_Erreur_Angulaire
        Vitesse_Angulaire = u2*(cos_Erreur_Angulaire**2) + w_r
        Vitesse_Lineaire = min(Vitesse_max, Vitesse_Lineaire)

        Vitesse_Gauche = Vitesse_Lineaire - L * Vitesse_Angulaire
        Vitesse_Droite = Vitesse_Lineaire + L * Vitesse_Angulaire
```

```
        sim.setJointTargetVelocity(moteur_droit2, Vitesse_Droite)
        sim.setJointTargetVelocity(moteur_gauche2, Vitesse_Gauche)

        time.sleep(Interval_Echantillonage)
        Temps=sim.getSimulationTime()

    # Arrêt des robots
    sim.setJointTargetVelocity(moteur_droit2, 0)
    sim.setJointTargetVelocity(moteur_gauche2, 0)
    sim.stopSimulation()

except KeyboardInterrupt:
    sim.setJointTargetVelocity(moteur_gauche2, 0)
    sim.setJointTargetVelocity(moteur_droit2, 0)
    sim.stopSimulation()
    print("Simulation interrompue.")
```

# Référence


- [Bayle, 2016] B. Bayle. : Robotique Mobile. University of Strasbourg, France, 2016.
- www.researchgate.net
- Commande Neuronale d'un Robot Mobile Non-Holonome avec glissement des roues- Salah Eddine Ghamri)